SPRINT 2 · DEEP LEARNING & COMPUTER VISION

Road Damage Object Detection

Training, comparison and deployment of YOLOv8s, YOLO11s and YOLO26s for detecting cracks, potholes and manholes in road images

Best model	Test mAP50 / 50-95	GPU speed (FP16)	Deployment
YOLO26s	**0.654 / 0.376**	**53–84 FPS**	**FastAPI**
highest accuracy	429 unseen images	RTX 3060 Laptop	+ web dashboard

Contents

1. Objective
2. Dataset and preparation
3. Models and training setup
4. Evaluation protocol
5. Results
6. Comparative analysis and discussion
7. Deployment application
8. Conclusion
9. Appendix: project structure and how to run

Prepared by: Karim Elroby · **Program:** INSTANT AI & Data Analysis Training · **Date:** 29 October 2026

1. Objective

Build, compare and deploy an AI system that detects and localizes road damage in images using YOLO-based object detection. The project covers:

- Training three YOLO generations — **YOLOv8s**, **YOLO11s** and **YOLO26s** — on the same data with the same settings.
- Comparing them on **mAP**, **precision / recall**, **IoU**, **inference speed (FPS)**, **model size** and **deployment efficiency** (ONNX export).
- A comparative analysis of the results.
- A deployment application (**FastAPI** backend + web dashboard) for image-based and real-time detection.

Detected classes: 🟦 **Crack**, 🟥 **Manhole**, 🟩 **Pothole**. Manholes are not damage, but detecting them explicitly prevents the model from confusing round manhole covers with potholes.

2. Dataset and preparation

The data is managed in a Roboflow project (`project-dvd12/road-damage-eul7g`). It combines road-damage images (cracks, potholes, manholes) with an additional set of manhole images to balance the classes. The project holds **4,643 images** and **6,323 bounding boxes**.

2.1 Data cleaning

- **Unlabeled images removed.** The dataset health check reported **802 "null" images** (no annotations). Manual inspection showed that most of them actually contain visible manholes and potholes that were never labeled. Training on them would teach the model that these objects are background, so they were tagged `unlabeled` and excluded with a *Filter by Tag* step (not deleted).
- **Class names unified.** `manholes` was renamed to `Manhole` (2,116 labels) to match `Crack` and `Pothole` .
- **No missing annotations** remained in the final version.

2.2 Dataset versions

Version	Resize	Offline augmentation	Split (train / valid / test)	Decision
v1	Stretch to 512×512	×3 (flip, brightness, noise)	3,957 / 352 / 87	Rejected: stretching distorts crack/pothole shapes; test set too small (87)
v2	Fit (black edges) 640×640	×3 (flip, brightness, noise)	≈3,700 / 352 / 175	Rejected: offline augmentation duplicates images and overlaps with YOLO's online augmentation
v3 (final)	**Fit (black edges) 640×640**	**None** (online augmentation in training)	**2,592 / 820 / 429**	**Used for all models**

Fit keeps the aspect ratio (letterboxing), so thin cracks are not stretched. Augmentation is applied on the fly by Ultralytics during training instead (mosaic, flips, HSV, scale, translate), which produces different variants every epoch.

2.3 Final dataset (v3)

Split	Images	Boxes	Crack	Manhole	Pothole
Train	2,592	4,243	1,601	1,443	1,199
Valid	820	1,290	425	481	384
Test	429	790	267	192	331
Total	**3,841**	**6,323**	**2,293**	**2,116**	**1,914**

The three classes are reasonably balanced (30–36% of boxes each). The split is fixed inside the Roboflow version, so all three models saw exactly the same training, validation and test images.

3. Models and training setup

Model	Generation	Params (M)	GFLOPs @640	Pretrained weights
YOLOv8s	Ultralytics YOLOv8	11.14	28.6	COCO `yolov8s.pt`
YOLO11s	Ultralytics YOLO11	9.43	21.7	COCO `yolo11s.pt`
YOLO26s	Ultralytics YOLO26	9.95	22.7	COCO `yolo26s.pt`

All models use the same size variant (**s**) so the comparison reflects the architecture, not the model size. They were fine-tuned from COCO weights with an **identical configuration**:

Setting	Value
Epochs / early stopping	100 / patience 20
Image size · batch	640 · 16
Optimizer	AdamW, lr0 = 0.001, lrf = 0.01
Weight decay · warm-up	0.0005 · 3 epochs
Seed · deterministic	42 · yes

Online augmentation	Value
Mosaic (closed last 10 epochs)	1.0
Horizontal flip	0.5
HSV hue / sat / value	0.015 / 0.7 / 0.4
Scale · translate	0.5 · 0.1
Training hardware	NVIDIA Tesla T4 (Kaggle / Colab)

Training notebooks (`notebooks/`): `road_damage_yolov8s.ipynb` , `road_damage_yolo11s.ipynb` , `road_damage_yolo26s.ipynb` (identical except for the model name). The best checkpoint (`best.pt` , highest validation fitness) of each run is used below.

4. Evaluation protocol

To make the comparison fair, all three `best.pt` files were evaluated with one script (`evaluation/evaluate.py`) on the **same 429 test images** (never used for training or model selection) and on the **same machine**: NVIDIA GeForce RTX 3060 Laptop GPU (6 GB) and Intel Core i5-11400H CPU, PyTorch 2.14 + CUDA 13.0.

Metric	How it is measured
mAP50, mAP50-95	Ultralytics validation on the test split (IoU threshold 0.5, and averaged over 0.5–0.95)
Precision, Recall	Mean over classes at the confidence that maximizes F1
Mean IoU	Predictions (conf ≥ 0.25) matched greedily to the same-class ground-truth box with the highest IoU (≥ 0.5); average IoU of these true positives = localization quality
Inference speed / FPS	Batch size 1, average over all test images (pre-process + inference + post-process). GPU in FP16, CPU in FP32, 10 warm-up images
Model size	`best.pt` file size, number of parameters, GFLOPs at 640×640
Deployment efficiency	Export to ONNX; ONNX file size, CPU speed with ONNX Runtime, and test mAP of the ONNX model (accuracy kept after export)

5. Results

5.1 Overall comparison

Model	mAP50	mAP50-95	Precision	Recall	Mean IoU	GPU FPS	GPU ms	CPU FPS	ONNX CPU FPS	Size (MB)
YOLOv8s	0.622	0.346	0.704	0.599	0.782	**84**	**11.9**	9.2	7.9	22.5
YOLO11s	0.636	0.361	0.693	**0.615**	0.784	66	15.2	**10.0**	**9.4**	**19.2**
YOLO26s	**0.654**	**0.376**	**0.761**	0.605	**0.790**	53	18.9	9.3	9.3	20.3

Bold = best value per column. GPU = RTX 3060 Laptop, FP16, batch 1. CPU = Intel Core i5-11400H.

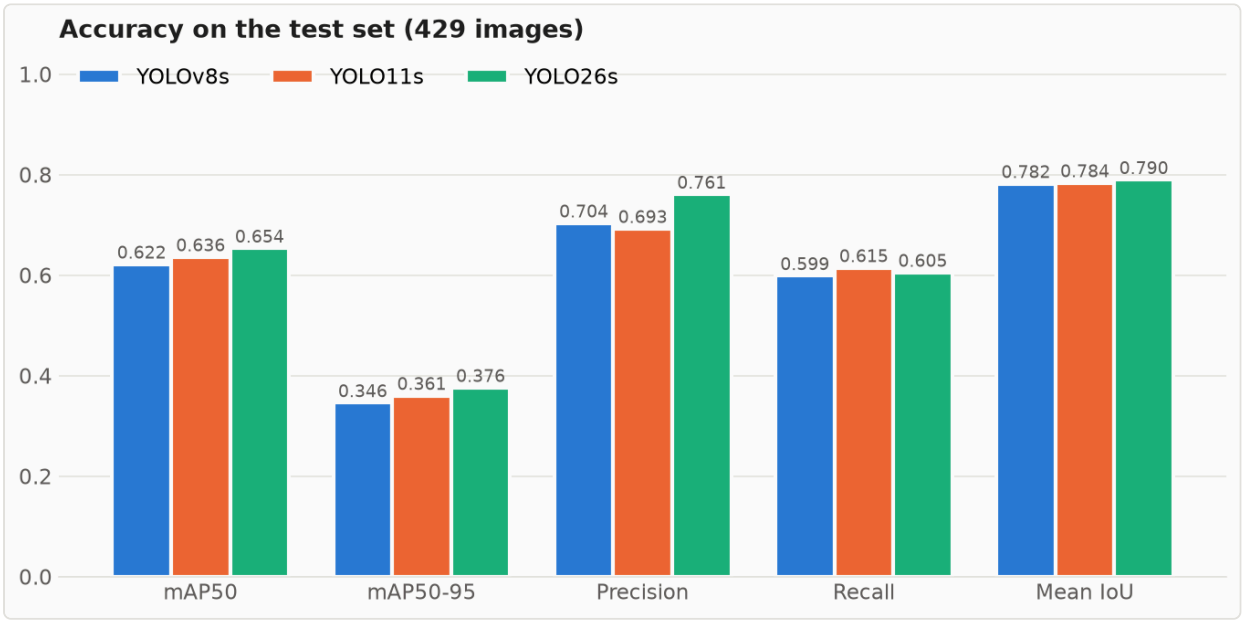


Figure 1 — Accuracy metrics on the test set (429 images).

5.2 Per-class results

Model	Crack			Manhole			Pothole		
	AP50	AP50-95	Recall	AP50	AP50-95	Recall	AP50	AP50-95	Recall
YOLOv8s	0.292	0.136	0.303	0.892	0.578	0.862	0.681	0.324	0.631
YOLO11s	0.308	0.148	0.307	**0.902**	0.602	**0.870**	0.699	0.332	**0.668**
YOLO26s	**0.355**	**0.167**	**0.333**	0.899	**0.606**	**0.870**	**0.707**	**0.356**	0.613

Bold = best per column. Precision per class (Crack / Manhole / Pothole): YOLOv8s 0.585 / 0.825 / 0.701 · YOLO11s 0.560 / 0.832 / 0.686 · YOLO26s 0.628 / 0.865 / 0.790.

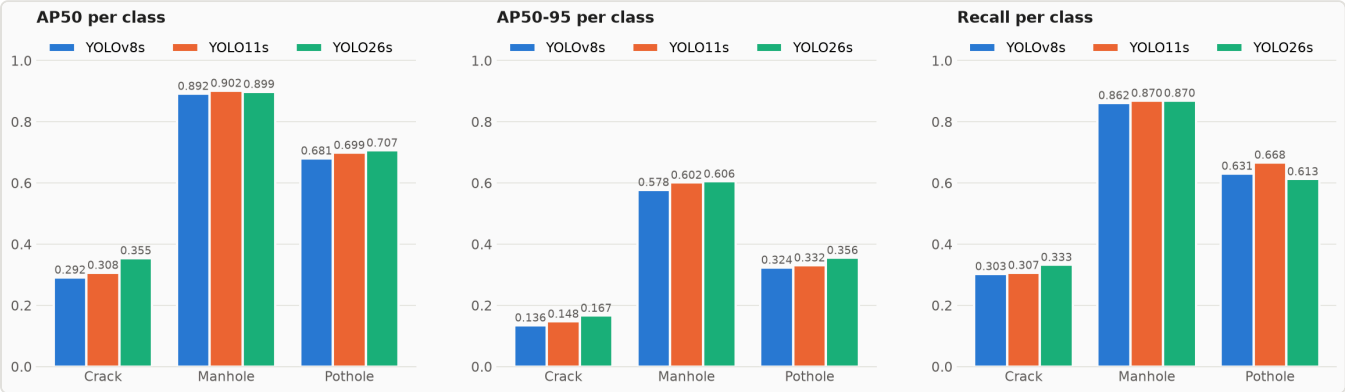


Figure 2 — AP50, AP50-95 and recall per class.

5.3 Speed, size and deployment efficiency

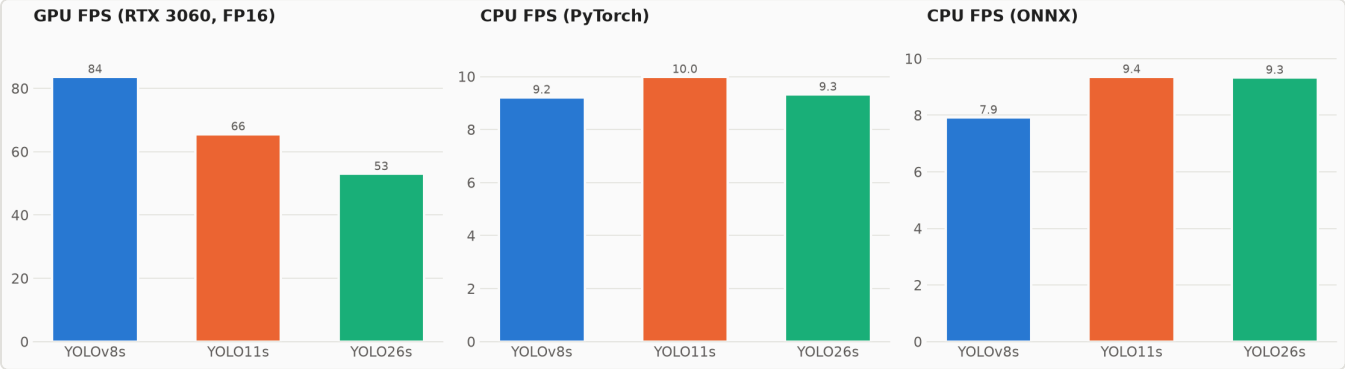


Figure 3 — Inference speed (batch 1): GPU FP16, CPU PyTorch and CPU ONNX Runtime.

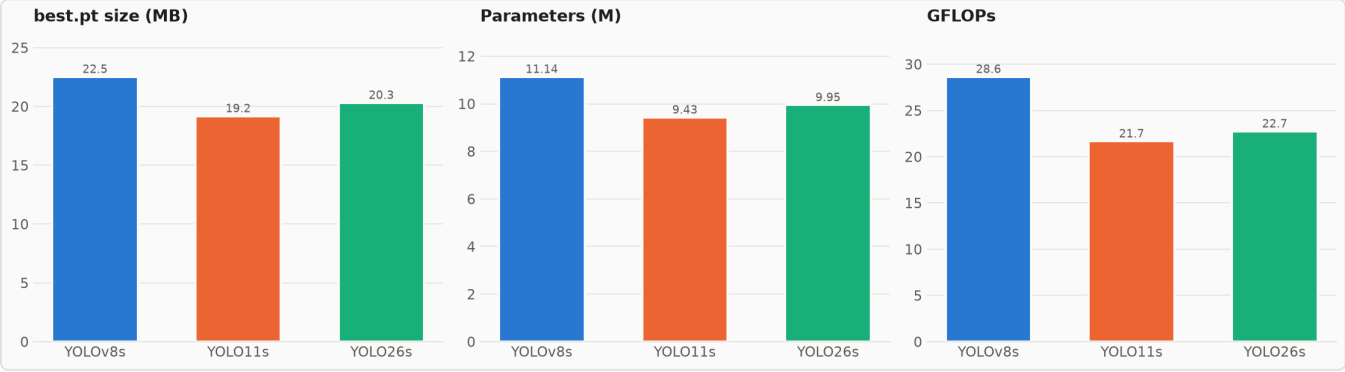


Figure 4 — Model size: file size, parameters and GFLOPs.

Model	GPU latency (pre / infer / post ms)	CPU ms	ONNX size (MB)	ONNX CPU ms	ONNX mAP50-95	Drop vs PyTorch
YOLOv8s	1.0 / 9.2 / 1.7	108.5	44.7	126.2	0.338	−0.008
YOLO11s	1.3 / 12.1 / 1.8	100.1	37.9	106.8	0.356	−0.005
YOLO26s	1.3 / 15.7 / 1.9	107.2	38.2	107.1	**0.374**	**−0.002**

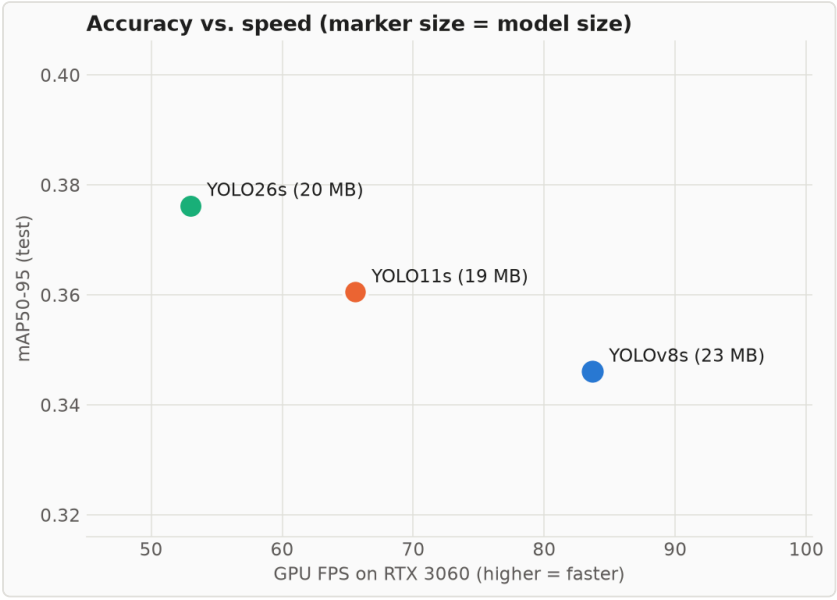


Figure 5 — Accuracy vs. GPU speed (marker size = model size). Top-right is best.

5.4 Confusion matrices (normalized)

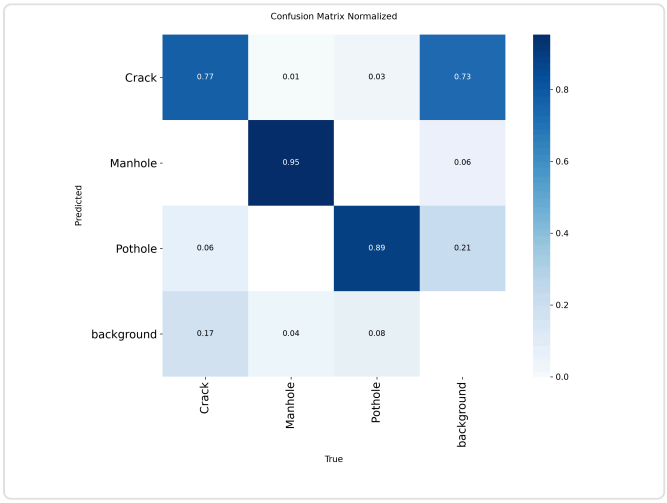


Figure 6 — YOLO26s.

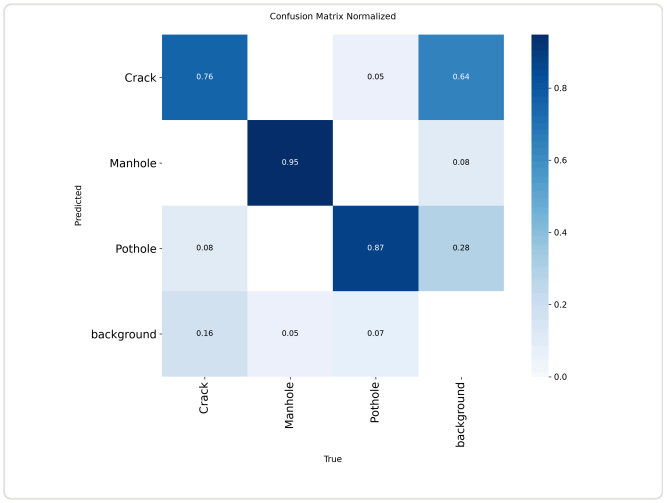


Figure 7 — YOLOv8s.

Columns are true classes, rows are predictions. The "background" column contains false positives, the "background" row contains missed objects. For all models most false positives are cracks, and cracks are also the most frequently missed class.

5.5 Qualitative results

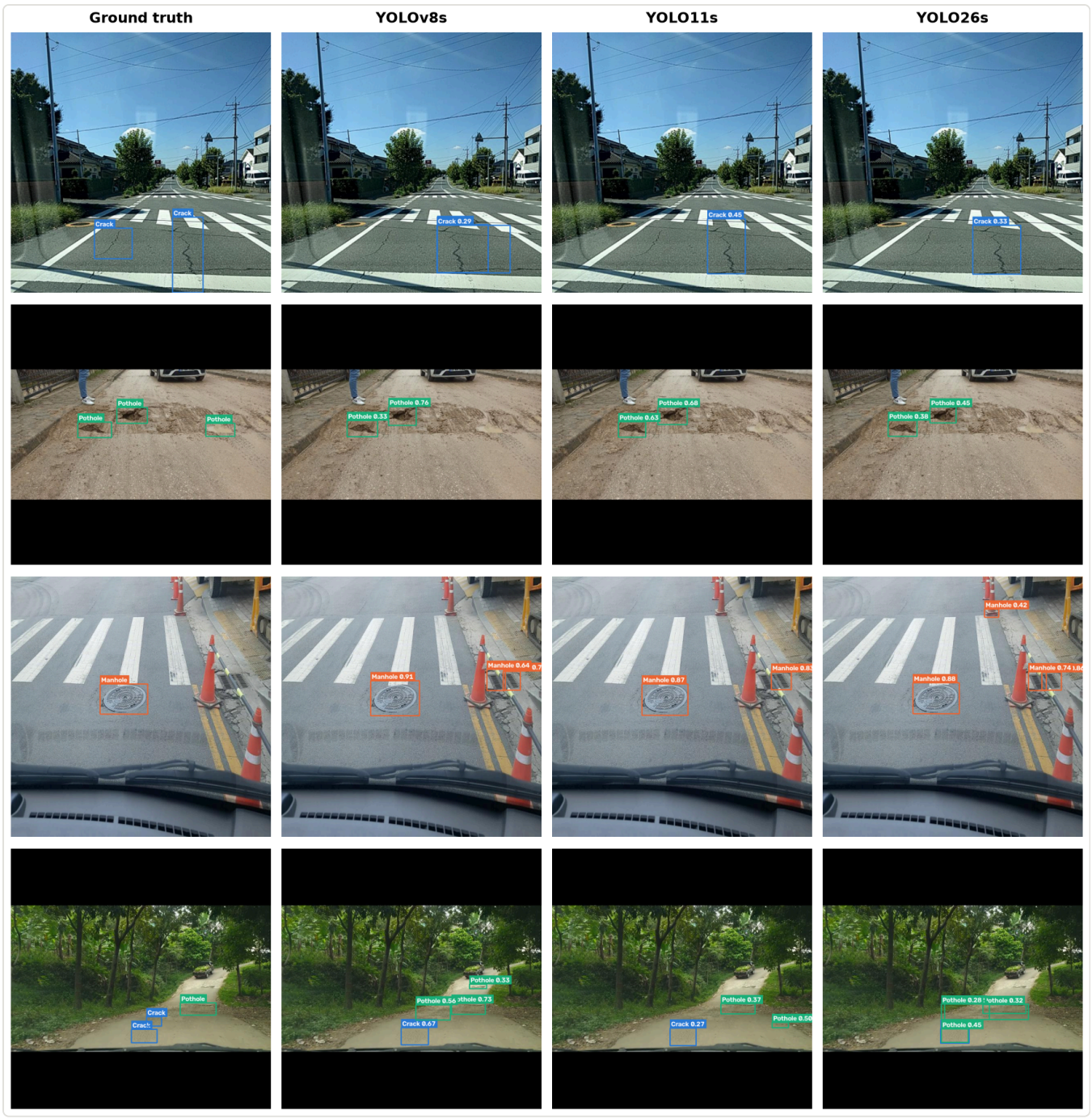


Figure 8 — Ground truth vs. predictions (conf ≥ 0.25) on four test images: crack, potholes, manhole, mixed scene.

6. Comparative analysis and discussion

6.1 Accuracy

- **The newer the generation, the more accurate:** YOLO26s > YOLO11s > YOLOv8s on mAP50 (0.654 / 0.636 / 0.622), mAP50-95 (0.376 / 0.361 / 0.346) and mean IoU (0.790 / 0.784 / 0.782). The ranking is consistent across all accuracy metrics, although the gaps are moderate (+3.2 mAP50 points from YOLOv8s to YOLO26s).
- **YOLO26s is clearly the most precise** (0.761 vs ≈0.70): it produces fewer false alarms at a similar recall (≈0.60–0.62). For a road-inspection tool this means fewer wasted inspections.
- **Localization is similar for all models** (mean IoU ≈0.78–0.79): when a damage is found, the box is tight in every case; the differences come mainly from *what* is found.

6.2 Per-class behaviour

- **Manhole is easy** (AP50 ≈0.89–0.90, recall 0.87): manhole covers have a distinctive round shape and texture.
- **Pothole is moderate** (AP50 ≈0.68–0.71): the boundary of a pothole is not well defined, which lowers AP50-95.
- **Crack is the hardest class** (AP50 0.29–0.36, recall ≈0.30–0.33). Cracks are thin, long and low-contrast; a single crack may be annotated as one box in one image and as several pieces in another; and an axis-aligned box around a diagonal crack is mostly background. These effects make both detection and the IoU-based AP50-95 difficult. **YOLO26s gives the largest improvement exactly on this class** (+6.3 AP50 points over YOLOv8s).
- **Confusions between classes are rare** (≤7%); the dominant error is crack vs. background, not crack vs. pothole.

6.3 Speed and size

- **All three models run in real time on the GPU** (53–84 FPS in FP16, batch 1). **YOLOv8s is the fastest** (11.9 ms per image) despite having the most parameters and FLOPs; the newer architectures use extra attention-style blocks that improve accuracy but add latency at batch size 1.
- **On the CPU the models are almost equal** (≈9–10 FPS). YOLO11s is the lightest model (9.4 M parameters, 19.2 MB) and the fastest on the CPU.
- **ONNX export keeps the accuracy** (drop ≤ 0.008 mAP50-95; only 0.002 for YOLO26s), so every model can be deployed without PyTorch. On this CPU, ONNX Runtime was not faster than PyTorch; its benefit is portability (no PyTorch dependency, runs on edge runtimes).

6.4 Model selection

A weighted score combines the requirements for a deployment app (min-max normalized; mAP50-95 40%, ONNX CPU speed 20%, recall 15%, mean IoU 10%, model size 10%, GPU speed 5%):

Model	Weighted score	Rank
YOLO26s	**0.824**	1
YOLO11s	0.683	2
YOLOv8s	0.050	3

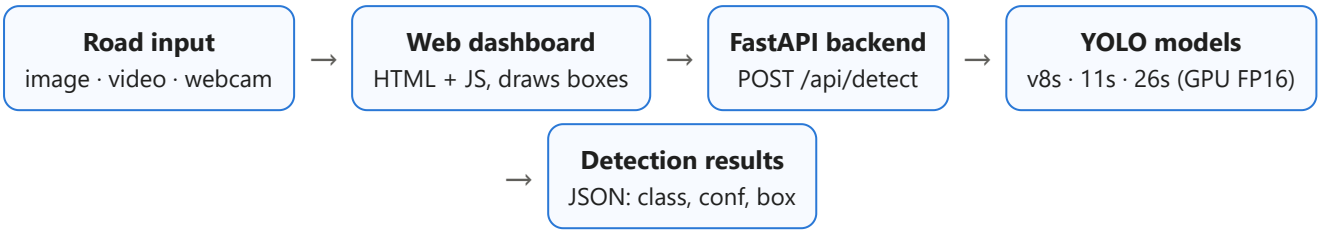
> **Selected model: YOLO26s.** It is the most accurate (highest mAP, precision and IoU, and the best crack detector), keeps its accuracy after ONNX export, and is still real-time on the GPU (53 FPS). It is the default model of the deployment app.
> **When to choose another model:** YOLOv8s when maximum GPU throughput matters more than accuracy (e.g. many camera streams); YOLO11s for CPU-only or memory-constrained devices (smallest and fastest on CPU). All three are available in the app.

6.5 Limitations

- **Label noise.** The data was merged from several sources. Some drains and manholes are still unlabeled (Figure 8, third row: the models detect drain grates that have no ground-truth box), so part of the measured "false positives" are correct detections. About 6% of the test images (25 / 429) are stock photos or illustrations rather than real road images.
- **Crack annotations are inconsistent** (one box vs. several boxes per crack), which limits the achievable crack AP for every model.
- **Test-set size.** With 429 test images, differences of about 0.01 mAP between models are within normal variation; the overall ranking is consistent across metrics, but small gaps should not be over-interpreted.

7. Deployment application

The selected models are served by a **FastAPI** backend with a web dashboard (folder `road_damage_app/`).



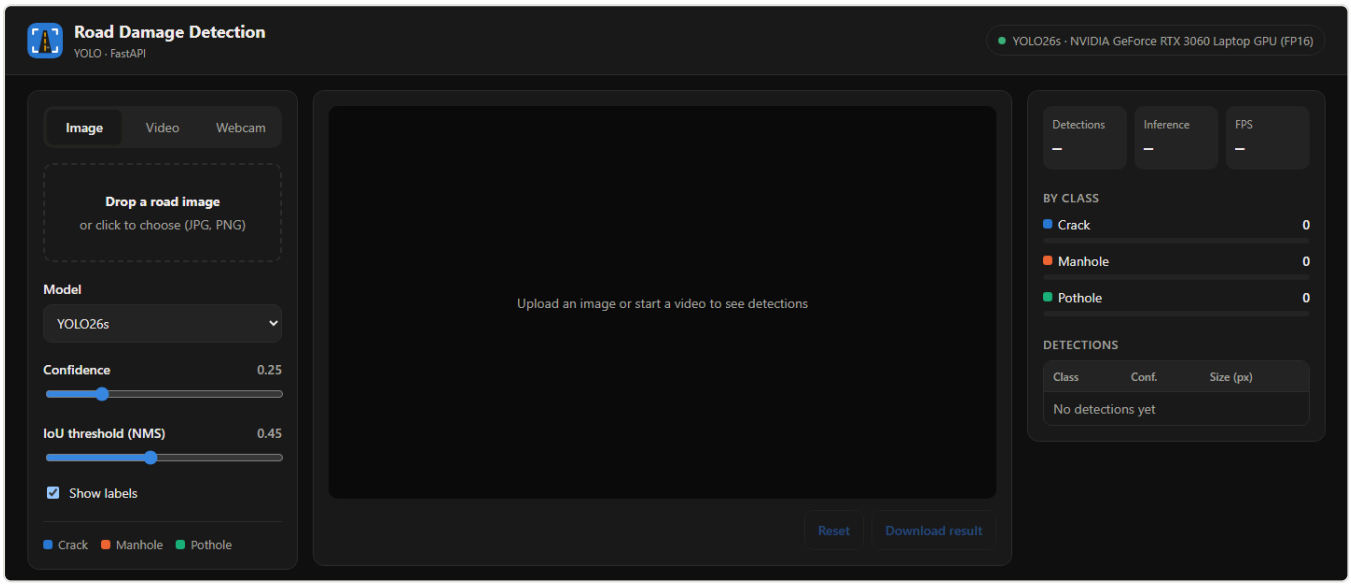


Figure 9 — Web dashboard (model selector, confidence and IoU sliders, live statistics).

Backend (FastAPI)

- Loads **all three models** once at start-up on the GPU (FP16), or on the CPU if no GPU is present; `.onnx` files are supported too.
- `GET /api/models` — available models and the default (YOLO26s).
- `POST /api/detect` — image + `model`, `conf`, `iou`, `imgsz`; returns boxes, class, confidence, counts per class and timing.
- Input validation (image type, 15 MB limit, parameter ranges), thread-safe inference, interactive API docs at `/docs`.
- 8 automated API tests (`pytest`), all passing.

Dashboard

- **Image mode:** drag-and-drop a road image; changing the model or the sliders re-runs detection instantly.
- **Video mode:** a local dashcam video is processed frame by frame in real time while it plays.
- **Webcam mode:** live camera detection.
- Statistics: detections, inference time, FPS, counts per class, detection table; download of the annotated result; reset.

On the RTX 3060 Laptop GPU, model inference takes about **12–19 ms per image** (Table 5.3); a full request including upload and decoding takes roughly 50–90 ms in the browser, which is fast enough for interactive and video use.

8. Conclusion

- Three YOLO generations were trained on the same cleaned dataset (3,841 images, 3 classes) with identical settings and evaluated on the same 429 unseen test images and hardware.
- **YOLO26s** is the most accurate model (mAP50 0.654, mAP50-95 0.376, precision 0.761) and the best on the hardest class (cracks); **YOLOv8s** is the fastest on the GPU (84 FPS); **YOLO11s** is the smallest and fastest on the CPU.
- All models are real-time on a consumer GPU and keep their accuracy after ONNX export.
- Cracks remain the main challenge (AP50 ≤ 0.36), mostly because of thin shapes and inconsistent annotations; cleaner and more consistent crack labels would give the biggest improvement.
- The FastAPI application makes the models usable for image-based and real-time road damage detection, with YOLO26s as the default.

Appendix: project structure and how to run

```
road-damage-detection/
├── notebooks/                 training notebooks (YOLOv8s, YOLO11s, YOLO26s) + comparison notebook
├── evaluation/
│   ├── download_test_data.py  downloads the Roboflow v3 test split
│   ├── evaluate.py            same-hardware evaluation of the 3 models (test split)
│   ├── compare.py             comparison table, charts, weighted ranking
│   ├── qualitative.py         ground truth vs. predictions figure
│   ├── results/               *_results.json, confusion matrices
│   └── comparison/            comparison.md, comparison_table.csv, figures
├── road_damage_app/           FastAPI backend + web dashboard
│   ├── app/main.py, app/detector.py, app/static/
│   ├── models/                trained weights: yolov8s.pt, yolo11s.pt, yolo26s.pt
│   └── run.py, tests/, requirements.txt, Dockerfile
├── report/                    this report
└── sample_test/               test images per class for quick checks
```

Run the application (Windows, from `road_damage_app`):

```
.venv\Scripts\python run.py        →  http://localhost:8000   (API docs: /docs)
```

Reproduce the evaluation (from `evaluation`):

```
..\road_damage_app\.venv\Scripts\python evaluate.py
..\road_damage_app\.venv\Scripts\python compare.py
```